The Space Complexity of Scannable Binary Objects

Sean Ovens
sgovens@cs.toronto.edu
University of Toronto
Toronto, Canada

ABSTRACT

Determining a consistent view of the contents of a shared array while its components are being modified is a fundamental task in distributed algorithm design. Snapshot objects address this problem when read/write registers are components of the array. A natural generalization of a snapshot object is a *scannable object*, an object with multiple components that each support *Read* and some other operations. Scannable objects also support the *Scan* operation, which provides an instantaneous view of all the object's components. This raises the question: How many instances of objects in some set O are required to implement a scannable object whose components are in O?

In this paper, we consider *scannable binary objects*, whose components are arbitrary 1-bit objects. If each component is a test-and-set object, there is a simple implementation using k test-and-set objects. However, when each component is non-monotonic (i.e. can change from 0 to 1 and from 1 to 0), we prove that more objects are needed. Specifically, when $n \leq 2^{k-1} + 2$, we show a lower bound of $n + k - 2$ base objects, even for obstruction-free, single-updater implementations. When $n \geq 2^k - 2$, we prove that $2^k - 1$ base objects are required for obstruction-free, single-updater implementations. When $2^{k-1} + 2 < n < 2^k - 2$, we show a gradual transition between these two lower bounds. We show that there is a lock-free implementation of an n-process, scannable binary object with k components from $n + k$ instances of 1-bit base objects, nearly matching our lower bound when $n \leq 2^{k-1} + 2$. There is a known wait-free, single-updater implementation that uses 2^k base objects, nearly matching our lower bound when $n \geq 2^k - 2$.

CCS CONCEPTS

• **Theory of computation → Distributed algorithms**.

KEYWORDS

space complexity, lower bound, snapshot object, shared memory

ACM Reference Format:
Sean Ovens. 2021. The Space Complexity of Scannable Binary Objects. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (PODC '21), July 26–30, 2021, Virtual Event, Italy*. ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3465084.3467916

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '21, July 26–30, 2021, Virtual Event, Italy
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8548-0/21/07…$15.00
https://doi.org/10.1145/3465084.3467916

1 INTRODUCTION

Scannable objects consist of multiple components that each support a *Read* operation as well as some other operations. Every component of the object can be read simultaneously using the *Scan* operation. The snapshot object [1, 2] is the well-studied special case of a scannable object in which each component is a register. It is known that k registers are necessary and sufficient to implement a snapshot object with k components. Many known implementations of scannable objects (including snapshot objects) use base objects that are large enough to store the result of a *Scan* [1, 3, 23, 25, 26]. If m is the domain size of each component, then a base object must have a domain size of at least m^k to store the result of a *Scan*. Hence, these implementations are impractical for scannable objects with many components, even if the components are small. Other implementations of scannable objects either use unbounded sequence numbers [1, 9, 15, 20], or require significantly more than k base objects [5, 8, 24, 33, 36]. It is natural to ask how much extra space (i.e. in addition to the space required to store the values of the object's components) is required to implement a scannable object. One way to investigate this is to consider implementations of scannable objects from base objects that are only large enough to store the value of a component. In this paper, as a starting point for this investigation, we consider implementations of *scannable binary objects*, whose components are arbitrary 1-bit objects, in a system that provides arbitrary 1-bit base objects.

We introduce a new approach for obtaining space lower bounds. We use our approach to obtain a lower bound on the number of base objects required to implement a scannable binary object for n processes with k non-monotonic components (i.e. the value of each component can be changed from 0 to 1 and from 1 to 0). We show how to construct an execution such that, for any extension of this execution in which the object never contains any vector of values in some set $\mathscr{V}$, there are $|\mathscr{V}|$ base objects that are not changed during this extension as long as $n - |\mathscr{V}|$ specific processes do not take steps. This allows us to obtain an implementation of a scannable binary object with $k - 1$ components for $|\mathscr{V}|$ fewer processes, using $|\mathscr{V}|$ fewer base objects. We use this to show that when $n \leq 2^{k-1} + 2$, at least $n + k - 2$ instances of 1-bit base objects are required to implement an n-process scannable binary object with k non-monotonic components. When $2^k - 2^{k-r} < n - 2 \leq 2^k - 2^{k-r-1}$, we show that $n + k - r - 2$ base objects are necessary. The lower bounds apply to obstruction-free, single-updater implementations. Every lock-free implementation is also obstruction-free, and every multi-updater implementation is also a single-updater implementation, which means that the lower bounds apply to lock-free, multi-updater implementations as well.

A single-updater scannable binary object with k components has a wait-free implementation from one k-bit single-writer register.

Furthermore, there is a wait-free implementation of a k-bit single-writer register from 2^k single-writer 1-bit registers [19, 32]. Thus, there is a wait-free implementation of a single-updater scannable binary object with k components from 2^k instances of 1-bit registers. When $n \geq 2^k - 2$, this nearly matches our lower bound. Thus, for this range, wait-free single-updater implementations are no more expensive than obstruction-free single-updater implementations.

There is a different restricted set of implementations that achieve better space complexity than our lower bound. Single-scanner objects allow only one process to perform *Scan* operations, while all of the other processes may only perform *Update* operations. In Section 4.1, we will show that there is a lock-free, single-scanner implementation of a scannable binary object with k components from $k + 1$ instances of 1-bit base objects.

Covering arguments are a standard approach to proving space lower bounds in distributed computing. A set of processes $\mathcal{P}$ *covers* a set of base objects $\mathcal{R}$ if, for all $R \in \mathcal{R}$, there is a process in $\mathcal{P}$ that is poised to apply a primitive that will overwrite the value of R in its next step. Covering arguments were originally used by Burns and Lynch [12] to prove that any mutual exclusion algorithm for $n \geq 2$ processes requires at least n shared registers. Since then, covering arguments have been used to prove space lower bounds for snapshots from registers [13], counters from swap objects [28], timestamps from registers [14, 21], and obstruction-free consensus from registers [34], among many other results [17]. Covering arguments can be used to obtain space lower bounds in systems that provide historyless base objects. If a set of $\mathcal{P}$ processes covers a set of historyless base objects $\mathcal{R}$, then allowing each of the processes in $\mathcal{P}$ to take their next steps (called a *block update* by $\mathcal{P}$) changes the objects in $\mathcal{R}$ so that they contain some fixed set of values. That is, we can use the block update by $\mathcal{P}$ to hide the information in $\mathcal{R}$ by overwriting the values in these base objects. Constructing a configuration in which the set of processes $\mathcal{P}$ covers the set of base objects $\mathcal{R}$ proves that the implementation requires at least $|\mathcal{R}|$ base objects. Since it is not possible to cover more than n base objects, covering arguments only allow us to obtain a lower bound of n at best. Achieving a lower bound of $n+k$ base objects requires different techniques. Furthermore, as we will discuss in Section 3, there are 1-bit objects that are not historyless. Hence, in our model, a set of covering processes cannot be used to overwrite the information in the set of covered base objects (since the effects of an operation can depend on the value of the base object).

We briefly survey related work in Section 2 and formally describe our model of computation in Section 3. In Section 4 we present some lock-free implementations of scannable binary objects to provide context for our lower bounds. Section 5 contains our lower bounds on the space complexity of scannable binary object implementations. We conclude with a brief discussion of some future research directions in Section 6.

2 RELATED WORK

Snapshot objects are fundamental for distributed algorithm design. The *Scan* operation of a snapshot object gives a process a view of the entire object at some point during the operation's execution interval, even while other processes are concurrently modifying the object. For this reason, snapshot objects have proved useful for simplifying the description of many algorithms in distributed computing, including obstruction-free consensus [11], bounded timestamping [18], and constructions of other large classes of objects [6, 22].

There are known lock-free implementations of an n-process, k-component snapshot object from k registers. Perhaps the simplest and most well-known of these implementations is the *double-collect* algorithm [1]: Each register is used to store the value of one of the components. An *Update* operation to component i simply writes to the register corresponding to component i. A *Scan* operation repeatedly collects the values in the components (i.e. reads each register one at a time) until the same vector of values is obtained twice in a row. This algorithm works as long as no register can change from value v to some different value v' and then eventually back to v. This sequence is called an ABA. One way to avoid this problem is to store a sequence number locally at each process, which increases with each *Update* performed by that process. An *Update* operation to component i by process p_j writes the triple (v, j, seq_j) to the base register corresponding to component i, where seq_j is p_j's current sequence number. This way, when a scanning process obtains the same vector from two consecutive collects, it must be true that at some point between these two collects, the snapshot object actually contained that vector. Guerraoui and Ruppert [20] showed that it is possible to remove the process identifier field from this implementation by obtaining more consecutive identical collects during a *Scan* operation. However, their implementation still uses unbounded sequence numbers. Both of these implementations are lock-free. It is possible for a scanning process to be delayed indefinitely by an infinite number of concurrent *Update* operations.

Some implementations of snapshot objects use *handshaking* bits rather than sequence numbers in order to bound the space required by the implementation. Handshaking is a standard technique used to coordinate pairs of processes [10, 30]. For example, the wait-free snapshot implementation of Afek et al. [1] uses 1-bit handshaking registers in order to allow *Scan* operations to detect concurrent *Update*s. A scanning process begins its operation by setting all of its handshaking bits to be equal to the corresponding bits that belong to the other processes. An updating process begins its operation by setting all of its handshaking bits to be the opposite value of the corresponding bits that belong to the other processes. Hence, if a scanning process p_j sees that one of its handshaking bits is not equal to the corresponding bit belonging to another process p_ℓ, then p_j knows that p_ℓ began an *Update* since p_j's handshaking loop. There are other more complicated techniques that can be used to obtain a wait-free implementation of a snapshot object. However, all of these implementations use registers large enough to store a view of the entire snapshot object [1, 9, 25], or require significantly more than k base registers [5, 8, 24, 36].

Single-scanner snapshots have received some attention [16, 27, 29, 31] due to their applications in garbage collection and checkpointing, for example. Fatourou and Kallimanis [16] present a single-scanner, n-process, k-component snapshot implementation that uses $k + 1$ unbounded registers. They also present a bounded-space implementation that uses $n + 1$ instances of $O(\log n)$-bit registers and $k \cdot (n + 3)$ instances of $O(\log m)$-bit registers, where m is the domain size of the components.

As mentioned previously, any single-updater scannable binary object with k components can be implemented with one instance of a k-bit single-writer multi-reader register. Vidyasankar [32] describes an implementation of a k-bit single-writer register from 2^k single-writer binary registers. A $Write$ of value v requires $v+1$ writes to the base objects, while a *Read* requires $\Theta(2^k)$ reads of the base objects in the worst case. There are faster implementations of single-writer registers with higher space complexity, like the implementations of Chen and Wei [32].

Scannable objects, just like snapshot objects, are helpful for simplifying the description of distributed algorithms. For example, Aspnes, Attiya, Censor-Hillel, and Ellen [5] defined the 2-*component max array*, which they used to implement an efficient snapshot algorithm assuming there are not too many *Update* invocations. The 2-component max array is a scannable object whose components are bounded *max-registers* [4], which support the $MaxWrite$ and $MaxRead$ operations. The $MaxRead$ operation returns the largest value written by a previous $MaxWrite$.

Ellen, Gelashvili, Shavit, and Zhu [15] classified various objects based on the number of instances of these objects required to solve n-valued obstruction-free consensus. In several cases, this task was solved using a scannable object. For instance, it was shown that n-valued obstruction-free consensus can be solved using a scannable 2-component max-register or a scannable $(n-1)$-component swap object. All of the scannable objects used in [15] can be implemented using the double-collect algorithm. Zhu [35] notes that for objects that can be augmented with sequence numbers like registers and swap objects, we can use the approach of Guerraoui and Ruppert [20] to obtain a lock-free implementation of a scannable k-component object from k base objects.

Jayanti [26] described an implementation of a scannable object $[O_1, \ldots, O_k]$ (which they call a *generalized* snapshot object in their paper) from an array of objects $A_1, \ldots, A_k$, where each A_i is the same type as O_i, along with a single LL/SC object large enough to store the result of a *Scan*. A *Scan* simply reads from the LL/SC object and returns the view that is obtained, and an $Update(i, op)$ operation applies op to A_i, collects the values in $A_1, \ldots, A_k$, and then tries to write this view to the LL/SC object. They also define a generalization of *Scan* that we call f-*Scan*. Instead of returning the vector of values present in the object, an f-*Scan* returns the result of applying the function f to the view obtained from a *Scan*. If f is the identity function, then an f-*Scan* is the same as a *Scan*.

Wei et al. [33] describe a technique for implementing a *Scan* operation for a collection of CAS objects. The technique involves augmenting each CAS object with an unbounded version list. A CAS operation obtains a timestamp from a global counter and (if the operation is successful) adds the timestamp paired with the new value to the head of the associated version list. A *Scan* can be performed by reading the value of the global counter to obtain some value t and finding a node in the version list of each CAS object whose timestamp does not exceed t.

Ellen, Fatourou, and Ruppert [13] proved that any n-process implementation of a k-component snapshot object requires at least k registers when $n \geq k$. Jayanti, Tan, and Toueg [28] proved that *single-writer* snapshot implementations (i.e. snapshot objects with n components, where each component can only be written by a single process) require at least $n-1$ registers. To our knowledge, there are no known space complexity lower bounds for scannable multi-component objects whose components support instructions other than *Read* and *Write*.

Initial value	0	1	0	1	
	Response		Resulting value		Historyless
Nop	$\perp$	$\perp$	0	1	✗
Read	0	1	0	1	✗
Write(1)	$\perp$	$\perp$	1	1	✓
test-and-set	0	1	1	1	✓
Write(0)	$\perp$	$\perp$	0	0	✓
test-and-reset	0	1	0	0	✓
flip	$\perp$	$\perp$	1	0	✗
test-and-flip	0	1	1	0	✗

Table 1: The eight invocations that can be supported by a 1-bit object. The pair of columns underneath "Response" give the responses to each invocation when they are applied to an object with value 0 or 1, where $\perp$ denotes the empty response. The pair of columns under "Resulting value" give the value of the object after applying each invocation when the object has value 0 or 1. The column under "Historyless" indicates which invocations are historyless.

3 MODEL

We use a standard asynchronous shared memory model in which a set of n processes $\{p_0, \ldots, p_{n-1}\}$ communicate using instances of shared *objects*. An object has a set of possible *values*, a set of *invocations* that can be applied, and a set of *responses* that can be returned. Each object has a *sequential specification*, which defines the response and the resulting value of the object, for each invocation *Inv* and value v. *Base objects* are provided by the system.

A *register* is an object that supports the following invocations: The *Read* invocation returns the value of the register and does not change its value. The $Write(v)$ invocation changes the value of the register to v and returns nothing.

An invocation is *trivial* if the application of the invocation does not change the value of the object, no matter what value the object has when the invocation is applied. We call an object *readable* if it supports the trivial *Read* invocation, which returns its value. A register is an example of a readable object.

A 1-*bit object* has two possible values, 0 and 1. There are eight possible invocations that can be supported by a 1-bit object, summarized in Table 1. An invocation is called *historyless* if the state of the object after the application of the invocation does not depend on the value of the object before the application of the invocation. Note that the response to a historyless invocation may depend on the value of the object. An object is historyless if every non-trivial invocation it supports is historyless. Note that the *flip* invocation is not historyless, since the value written by *flip* depends on the current value of the object. A 1-bit object is *non-monotonic* if it supports an invocation that changes its value from 0 to 1, and it supports an invocation that changes its value from 1 to 0.

Some objects may appear to be 1-bit objects at first glance, even though their domain size is actually much larger. For example, an

LL/SC bit stores a bit $v \in \{0, 1\}$ and supports two invocations: The *load-link* (*LL*) invocation returns v. A *store-conditional* invocation with input $v' \in \{0, 1\}$ ($SC(v')$) by process p_j returns either *True* or *False*, and may change the bit stored by the object. If there was some *SC* invocation that returned *True* since p_j's most recent *LL*, then p_j's $SC(v')$ invocation does not change the bit stored by the object and returns *False*. Otherwise, it changes the bit stored by the object to v' and returns *True*. Even though v is a single bit, the domain size of an LL/SC bit is much larger. In particular, the value of an LL/SC bit needs to track which processes have done an *LL* since the latest *SC* that returned *True*. This requires n bits of information. It is also worth noting that the *LL* invocation does not return the entire value of the object (it only returns the bit v). Hence, an LL/SC bit is not readable.

An *implementation* of an object O defines a set of base objects, a set of initial configurations, and an algorithm for each process to follow for each invocation supported by O. To avoid confusion, we will call the invocations supported by implemented objects *operations*, and we will call the invocations supported by base objects *primitives*. A *configuration* of an implementation consists of a state for every process and a value for every base object defined by the implementation. A *step* by a process consists of a primitive applied to some base object, a response to that primitive, and some finite amount of local computation that may change the state of the process. A step may also begin with an operation on the implemented object, and it may end with a response to an operation on the implemented object. An *initial configuration* of an implementation defines the values of the base objects in the implementation before processes have taken any steps.

An *execution* of an implementation is an alternating sequence of configurations and steps that begins with a configuration. A finite execution ends with a configuration. If C is a configuration and α is an execution from C, then we use $C\alpha$ to denote the final configuration in α. If $C' = C\alpha$, then we say C' is *reachable* from C via the execution α. An execution α is *P-only*, for some set of processes P, if α only contains steps by processes in P. If P contains just one process p_j, then we say α is p_j-only.

Suppose that C_0 is an initial configuration of an implementation $\mathcal{I}$. Let C be a configuration that is reachable from C_0 via the execution α. An *instance* of an operation on the implemented object is an occurrence of that operation in some step of α. If op is an instance of an operation in a step by process p_j, then we say op is *ongoing* in C if there is no response that appears after op in α at the end of a step by p_j. Each process can have at most one ongoing instance of an operation in any configuration. We say that two operation instances *overlap* if there is a configuration in which they are both ongoing. If an instance of an operation is not ongoing in C, then we say it is *complete*. If p_j does not have an ongoing instance of an operation in C, then we say C is p_j*-idle*. If P is some set of processes and C is p_ℓ-idle for every process $p_\ell \in P$, then we say C is P-idle.

An execution is produced by a *scheduler*, which decides the operations on the implemented object that processes perform, and the order in which processes take steps. In any initial configuration of an implementation, each process is idle. When an idle process p_j is scheduled to take a step, then this step begins with an instance of an operation on the implemented object, which is chosen by the scheduler. This instance of an operation is ongoing until p_j returns a response to it at the end of some step. When p_j takes a step, the primitive that it applies and the base object B to which this primitive is applied is determined by p_j's current state along with its algorithm for its ongoing instance of an operation, as defined by the implementation. The response to the primitive in p_j's step and the resulting value of B are defined by the sequential specification of B. After receiving a response to its primitive, p_j performs some finite amount of local computation and possibly updates its state. When p_j's algorithm terminates, it returns a response to its ongoing operation instance at the end of its step, updates its state, and then becomes idle again.

Let $\mathcal{I}$ be an implementation of an object O. An execution α from an initial configuration of $\mathcal{I}$ is called *linearizable* if and only if there exists a sequence Π of instances of operations and responses such that

(i) Π contains every complete operation instance in α, each immediately followed by its response, and a subset of the incomplete operation instances in α, each immediately followed by some response,
(ii) if the response to a complete instance op_1 of an operation appears before an instance op_2 of an operation in α, and op_2 appears in Π, then op_1 appears before op_2 in Π, and
(iii) the sequence Π satisfies the sequential specification of O. That is, if the operations in Π were applied to O sequentially, then the sequence of responses returned would be the same as the sequence of responses in Π.

We call Π a *linearization* of α.

Every complete instance of an operation in a linearizable execution α can be assigned a unique *linearization point* in the execution, at which the operation instance appears to take effect. This point must occur between the step containing the instance of the operation and the step containing its response. We say that the instance of the operation is *linearized* at this point. An instance of an operation that is incomplete in α may also be linearized at any point at or after the step containing the operation. If the operation instances that have been linearized in α are arranged in the order of their linearization points, then the resulting sequence obeys the sequential specification of the implemented object.

Two configurations C_1 and C_2 are *indistinguishable* to a process p_j if p_j has the same state in C_1 and C_2. This is denoted by $C_1 \overset{p_j}{\sim} C_2$. We use $val(B, C)$ to denote the value of the base object B in configuration C.

Let α_1 and α_2 be executions starting from the configurations C_1 and C_2, respectively. Then α_1 and α_2 are *indistinguishable* to process p_j if $C_1 \overset{p_j}{\sim} C_2$ and p_j performs the same sequence of steps in α_1 and α_2. This is denoted by $\alpha_1 \overset{p_j}{\sim} \alpha_2$. If $C_1 \overset{p_j}{\sim} C_2$ and $\alpha_1 \overset{p_j}{\sim} \alpha_2$, then $C_1\alpha_1 \overset{p_j}{\sim} C_2\alpha_2$. If β_1 is a p_j-only execution from C_1, the set of base objects accessed during β_1 have the same values in C_1 and C_2, and $C_1 \overset{p_j}{\sim} C_2$, then there exists a p_j-only execution β_2 from C_2 such that $\beta_1 \overset{p_j}{\sim} \beta_2$ [7, Corollary 2.2].

An implementation is *wait-free* if every instance of an operation on the implemented object performed by a process completes within a finite number of steps by that process. An implementation is *lock-free* if every infinite execution of the implementation

contains an infinite number of complete operation instances. An implementation is *obstruction-free* if every instance of an operation completes when the process performing the operation takes sufficiently many consecutive steps. Our lower bounds in Section 5 apply to obstruction-free implementations. Since all lock-free and wait-free implementations are also obstruction-free, this means that our lower bounds also apply to lock-free and wait-free implementations.

A *scannable* object is a sequence of readable objects $[O_1, \ldots, O_k]$. Each O_i is called a *component* of the object. A scannable object supports the $Update(i, op)$ operation, where op is an invocation supported by O_i, which applies the invocation op to O_i and returns the response of op. A scannable object also supports the *Scan* operation, which performs a simultaneous *Read* of each object in the sequence. A *scannable binary object* is a scannable object in which each component is a 1-bit object. For any implementation $\mathcal{I}$ of a scannable binary object $[O_1, \ldots, O_k]$, we assume that $\mathcal{I}$ has one unique initial configuration C_0 in which every component O_i has value 0. The value of a scannable binary object $[O_1, \ldots, O_k]$ is a k-component vector $V \in \{0, 1\}^k$.

An *s-scanner, u-updater* implementation $\mathcal{I}$ of a scannable object allows only s fixed processes, called the *scanners*, to perform *Scan* operations, and it allows only u fixed processes, called the *updaters*, to perform *Update* operations. Every process can be a scanner, an updater, or both. Note that this means $s + u \geq n$. If $s = 1$, we say that $\mathcal{I}$ is a *single-scanner* implementation. If $u = 1$, we say that $\mathcal{I}$ is a *single-updater* implementation. (Note that this is different from a *single-writer* scannable object [1, 2], for which component O_i can only be modified by process p_i. In a single-updater implementation, the updater can perform *Update* operations on any component.)

4 UPPER BOUNDS

In some cases, it is simple to implement a scannable binary object. For instance, consider a scannable binary object consisting of test-and-set objects. Once a process applies a *TAS* invocation to a test-and-set object, there is no way for the object to return to the value 0. In other words, test-and-set objects do not experience ABAs. Therefore, the simple double-collect algorithm suffices to implement a scannable object whose components are test-and-set objects. This means that we need only k instances of test-and-set base objects to implement a scannable binary object consisting of k test-and-set objects. In fact, this implementation is wait-free, since each component can only be changed once.

However, for scannable binary objects with non-monotonic components, we need a different approach. We cannot augment 1-bit base objects with process identifiers or sequence numbers. In fact, in Section 5, we will show that there is no obstruction-free implementation of a scannable binary object with $k \geq 2$ non-monotonic components from k instances of 1-bit base objects.

In this section, we present some implementations of an n-process scannable binary object $[O_1, \ldots, O_k]$ from 1-bit base objects. We begin by presenting a lock-free, s-scanner, u-updater implementation that uses $s+k$ instances of 1-bit base objects. In particular, this gives an n-scanner, n-updater implementation from $n + k$ base objects, and it gives a single-scanner implementation from $k+1$ base objects. The second implementation is wait-free, single-updater, and uses 2^k instances of 1-bit base objects. While the single-scanner and single-updater implementations are not particularly interesting on their own, they both use fewer than $n + k$ base objects for certain ranges of n. Therefore, a proof that $n + k$ base objects are necessary in the multi-scanner, multi-updater setting would need use executions that contain both overlapping *Update* and overlapping *Scan* operations.

4.1 A multi-updater, multi-scanner implementation

Our first implementation is based on the double-collect algorithm [1] described in Section 2. Since we are using 1-bit base objects, it is not possible to use process identifiers or sequence numbers to avoid the ABA problem. Our algorithm requires that a scanner obtains $u + 1$ consecutive identical collects, rather than just 2. This idea was also used by Guerraoui and Ruppert [20] to implement a snapshot object that does not use process identifiers. However, their implementation still uses sequence numbers, so we cannot use their algorithm directly. We instead use an array of *handshaking* bits to allow scanners to detect concurrent updates. We describe the algorithm in more detail below.

Implementation 1 is a lock-free, s-scanner, u-updater implementation of $[O_1, \ldots, O_k]$. Assume that the processes $p_0, \ldots, p_{s-1}$ are all scanners and $p_s, \ldots, p_{n-1}$ are all updaters. Note that some of $p_0, \ldots, p_{s-1}$ may be updaters as well. The implementation uses k instances of 1-bit base objects $B_1, \ldots, B_k$, where each B_i has the same type as component O_i, and s binary registers $R_0, \ldots, R_{s-1}$. The base objects $B_1, \ldots, B_k$ are used to store the values of the components $O_1, \ldots, O_k$, while the registers $R_0, \ldots, R_{s-1}$ are used for handshaking. An $Update(i, op)$ operation first sets all of the handshaking bits to 1 one at a time. This signals to ongoing instances of *Scan* operations that the object is about to be modified. The updating process then applies the primitive op to B_i and returns its response. A *Scan* operation by process p_j first collects the values in $B_1, \ldots, B_k$ and saves the resulting vector in the local variable b. Then it writes 0 to its handshaking bit R_j. Process p_j then repeatedly collects the values in $B_1, \ldots, B_k, R_j$. Once p_j reads b from $B_1, \ldots, B_k$ and reads 0 from R_j in $u + 1$ consecutive collects, it returns the vector b. If at any point p_j observes that the value of some base object B_i is different from the i-th component of b, or it reads a 1 from R_j, then p_j restarts its *Scan*.

A scanning process wants to perform an entire collect during which no *Update* operation changes the contents of any of the base objects $B_1, \ldots, B_k$. This way, the scanner can be certain that the vector obtained by this collect accurately reflects the contents of $B_1, \ldots, B_k$ throughout the execution interval of that collect. If a scanner reads a 1 from its handshaking bit at any point, then it knows that an instance of an *Update* operation has started since it last restarted its *Scan*. It is possible that an *Update* finishes setting all of the handshaking bits to 1 just before a scanning process resets its handshaking bit to 0. In this case, the *Update* might change one of the base objects $B_1, \ldots, B_k$ while the scanner is performing a collect. However, in Lemma 1 we will show that there must be at least one uninterrupted collect among the final $u + 1$ collects performed during a complete instance of a *Scan* operation.

Lemma 1. *Let sc be a complete instance of a* Scan *operation by process p_j in an execution of Implementation 1. Among the last $u+1$ collects performed by sc, there must be at least one collect during which no* Update *applied a primitive to any base object $B_1, \ldots, B_k$.*

Proof. Let $coll_0$ denote the first of these final $u+1$ collects and let $coll_u$ denote the last. The scanning process p_j checks its handshaking bit R_j immediately following each of these final n collects. If any of these *Read* primitives returns 1, then p_j restarts its *Scan*, performing at least $u+1$ more collects. Hence, it must be the case that each of these *Read* primitives on R_j returns 0. Only process p_j can write 0 to R_j. Furthermore, p_j writes 0 to R_j immediately before $coll_0$ and does not write to R_j for the remainder of *sc*. Thus, no *Update* writes 1 to R_j after the beginning of $coll_0$ and before the end of $coll_u$. An *Update* begins by setting each handshaking bit (including R_j) to 1 prior to modifying any of the base objects $B_1, \ldots, B_k$. Therefore, every instance of an *Update* operation that modifies some base object in $B_1, \ldots, B_k$ after the beginning of $coll_0$ and before the end of $coll_u$ must have set R_j to 1 before the beginning of $coll_0$. That is, the *Update* operation instance must have been ongoing at the beginning of $coll_0$. Hence, at most u instances of *Update* operations modify a base object in $B_1, \ldots, B_k$ after the beginning of $coll_0$ and before the end of $coll_u$. Thus, there was at least one collect performed by p_j in this interval during which no *Update* modified a base object in $B_1, \ldots, B_k$. □

Theorem 2. *Implementation 1 is a lock-free, linearizable, s-scanner, u-updater implementation of the scannable binary object $[O_1, \ldots, O_k]$ from $s+k$ instances of 1-bit base objects.*

Proof. We can linearize an instance of an $Update(i, op)$ operation when it applies op to B_i. Lemma 1 says that, among the last $u+1$ collects performed by a complete instance *sc* of a *Scan* operation, there is at least one collect during which no *Update* changed any of $B_1, \ldots, B_k$. Since *sc* returns the vector obtained from this collect, we can linearize *sc* at the beginning of this collect.

Every instance of an *Update* operation applies exactly $s+1$ primitives to base objects. A process p_j performing a *Scan* only restarts its operation instance when it observes a change in one of the base objects $B_1, \ldots, B_k$, or it sees a 1 in its handshaking bit R_j. Any instance of an *Update* operation writes 1 to R_j exactly once, and modifies a base object $B_1, \ldots, B_k$ at most once. Hence, if an instance of a *Scan* operation runs infinitely long without completing, an infinite number of concurrent *Update* operation instances must be completed. This implies that Implementation 1 is lock-free. □

4.2 A single-updater, multi-scanner implementation

We now consider the single-updater setting in which only p_0 is an updater. It is simple to implement the scannable binary object $[O_1, \ldots, O_k]$ from one k-bit single-writer register. Process p_0 keeps a local copy of the value of $[O_1, \ldots O_k]$. To perform an *Update*, p_0 modifies the appropriate bit of the value in its local copy and writes the new value to the register. A *Scan* by a process $p_1, \ldots, p_{n-1}$ reads the register and returns the obtained vector. A *Scan* by p_0 simply returns p_0's local copy.

There is a wait-free, linearizable, single-writer implementation of a k-bit register from 2^k single-writer 1-bit registers $R_0, \ldots, R_{2^k-1}$ [19, 32]. The register R_0 is initialized with the value 1, and all other registers are initialized with the value 0. An instance of a $Write(v)$ operation, for some $v \in \{0, \ldots, 2^k-1\}$, first writes a 1 to R_v, and then writes a 0 to each of the registers $R_{v-1}, \ldots, R_0$ in order. An instance of a *Read* operation reads the registers $R_0, \ldots, R_{2^k-1}$ in order until it reads a 1 from some register R_w. The reader saves the value w in a variable u and then reads all of the registers in $R_{w-1}, \ldots, R_0$ in order. Whenever the reader sees a 1 in the register $R_{w'}$ during this sequence of reads, it updates u to the value w'. Following this sequence, the reader returns u. Hence, there is a wait-free, linearizable, single-updater, n-process implementation of $[O_1, \ldots, O_k]$ from 2^k instances of 1-bit single-writer, multi-reader registers.

5 LOWER BOUNDS

In this section, we present our lower bounds on the number of arbitrary 1-bit base objects required to implement a scannable binary object with non-monotonic components. We show that when $k \geq 3$ and $2^k - 2^{k-r} < n - 2 \leq 2^k - 2^{k-r-1}$, any obstruction-free, linearizable, single-updater, n-process implementation of a scannable binary object with k non-monotonic components requires at least $n+k-r-2$ instances of 1-bit base objects. In particular, when $n-2 \leq 2^{k-1}$, this implies that $n+k-2$ instances of 1-bit base objects are required, nearly matching our $n+k$ upper bound in Section 4.1.

In most known implementations of scannable objects (including the implementation discussed in Section 4.1), the k components of the object $O_1, \ldots, O_k$ correspond to k base objects $B_1, \ldots, B_k$. That is, for each component O_i there exists a specific base object B_i that stores the value of O_i. We make no such assumption for the lower bounds given in this section.

All of our proofs in this section apply to the single-updater setting, where process p_0 is the only updater. This means that the lower bounds also apply to the general setting in which any process can perform *Update* operations.

Proving our lower bound for single-updater implementations simplifies our arguments. Since p_0 is the only process that can perform *Update* operations, there are no overlapping instances of *Update* operations in any configuration of a single-updater implementation. This means that for any execution α that ends with a p_0-idle configuration, there is only one way to linearize the *Update* operation instances in α. Then we can determine the value of component i at the end of α using the sequential specification of component i and the sequence of *Update* operation instances on component i in α. If component i is non-monotonic and it has value v in some p_0-idle configuration, there there is an operation on i that changes its value from v to $1-v$ (by definition of non-monotonic).

Let $\mathcal{I}$ be an obstruction-free, linearizable, single-updater implementation of a scannable binary object $[O_1, \ldots, O_k]$ with initial configuration C_0, and let C be some configuration reachable from C_0. Let $\mathcal{V}$ be a set of k-component binary vectors. Let α be an execution from C and let α' be the execution from $C\alpha$ in which p_0 completes its ongoing *Update* operation instance. (If $C\alpha$ is p_0-idle, then let α' be empty.) Then α is called $\mathcal{V}$*-absent* if and only if, for

every p_0-idle configuration C' reached during $\alpha\alpha'$, the value of the scannable binary object in C' is not equal to any vector in the set $\mathscr{V}$. If $\mathscr{V}$ contains just one vector V, we say that α is V-absent. If α is $\mathscr{V}$-absent, then an execution β from $C\alpha$ is called a *$\mathscr{V}$-absent extension* of α if and only if $\alpha\beta$ is $\mathscr{V}$-absent. Notice that if α is $\mathscr{V}$-absent, then α is also $\mathscr{V}'$-absent for any $\mathscr{V}' \subseteq \mathscr{V}$. Any execution from any configuration is $\emptyset$-absent.

The following observation is immediate from the definition of $\mathscr{V}$-absent:

Observation 3. *Let α be a $\mathscr{V}$-absent execution from configuration C. If sc is a complete instance of a* Scan *operation in α, then the response of sc is not equal to any vector in $\mathscr{V}$.*

Since scanners do not affect the values stored in the components of the scannable object, a $\mathscr{V}$-absent execution can always be extended with a sequence of steps by the scanners.

Observation 4. *Let α be a $\mathscr{V}$-absent execution from some configuration C. Any execution from $C\alpha$ in which only the scanners $p_1, \ldots, p_{n-1}$ take steps is a $\mathscr{V}$-absent extension of α.*

We will prove several lemmas about an arbitrary obstruction-free, linearizable, single-updater, n-process implementation $\mathcal{I}$ of a scannable binary object $[O_1, \ldots, O_k]$, where each component O_i is non-monotonic and $n, k \geq 2$. Let $\mathcal{B}$ be the set of 1-bit base objects used by $\mathcal{I}$, and let $m = |\mathcal{B}|$. Let C_0 be the initial configuration of $\mathcal{I}$.

In our first lemma, we show that there is a set of k-component binary vectors $\mathscr{V}$ and an execution α from C_0 such that, for any $\mathscr{V}$-absent execution from $C_0\alpha$ that does not involve the last $|\mathscr{V}|$ scanners, there is a set of $|\mathscr{V}|$ base objects that are not changed by any process. We use this lemma with a set $\mathscr{V}$ consisting of just one vector V to show that if $m = k$, then there are not enough values for the sequence of base objects to represent all of the possible values of $[O_1, \ldots, O_k]$ in a V-absent execution. This means that $m \geq k+1$. We use this fact in the base case of the proof of our main result.

Next, we show that if $k \geq 3$, we can effectively remove one component, h processes, and h base objects from the implementation $\mathcal{I}$ to obtain a new implementation $\mathcal{I}'$ (where $h = min(n-2, 2^{k-1})$). We can apply this repeatedly until we have a 2-process implementation of a scannable binary object with $k' < k$ components. This implementation must use at least $k'+1$ base objects. We use this to obtain a lower bound on the number of base objects required by the original k-component implementation.

To provide some intuition for the proof of the first lemma, we begin with an informal proof of a special case. Let V be an arbitrary k-component binary vector that contains a 1 in its k-th component. We want to show that there is an execution α from C_0 and a base object $B \in \mathcal{B}$ such that, in any V-absent execution from $C_0\alpha$ that does not involve p_{n-1} (i.e. a $\{p_0, \ldots, p_{n-2}\}$-only execution), the value of B is not changed. To derive a contradiction, suppose that such an execution does not exist. That is, suppose that for every execution α from C_0 and for every base object $B \in \mathcal{B}$, there is a $\{p_0, \ldots, p_{n-2}\}$-only, V-absent execution from $C_0\alpha$ that ends with a primitive that changes the value of B. Let μ be a p_0-only execution from C_0 in which p_0 changes the value of the scannable binary object to V. Now consider the p_{n-1}-only execution δ from $C_0\mu$ in which p_{n-1} does a complete *Scan*. By construction of μ, this *Scan* must return V.

We show how to construct a V-absent execution γ from C_0 that is indistinguishable to p_{n-1} from δ. Suppose that $B_1 \in \mathcal{B}$ is the first base object accessed by p_{n-1} in δ. If the value of B_1 in C_0 is the same as the value it had when p_{n-1} first accessed it during δ, then γ simply begins with the first step that p_{n-1} performed in δ. Otherwise, we know that there is a $\{p_0, \ldots, p_{n-2}\}$-only, V-absent execution β from C_0 that ends with primitive that changes the value of B_1. That is, the value of B_1 in $C_0\beta$ is the same as the value it had when p_{n-1} first accessed it during δ. Then γ begins with β followed by the first step that p_{n-1} performed in δ. We repeat this process for every step of δ to construct γ. Since p_{n-1}'s *Scan* operation in δ returns V, its *Scan* operation in γ must return V as well. This contradicts Observation 3. Hence, there is an execution α from C_0 and a base object $B \in \mathcal{B}$ such that, in any V-absent, $\{p_0, \ldots, p_{n-2}\}$-only execution from $C_0\alpha$, the value of B is not changed.

More generally, let $\ell = min(n-1, 2^{k-1})$, and let $\{V_1, \ldots, V_\ell\}$ be a set of ℓ distinct k-component binary vectors that all contain 1 in their k-th component. Note that it is possible to find such a set of vectors since $\ell \leq 2^{k-1}$. Define $\mathscr{V}_j = \{V_1, \ldots, V_j\}$ for all $j \in \{1, \ldots, \ell\}$, and define $\mathscr{V}_0 = \emptyset$. We have already argued that when $j = 1$, there is an execution α_j from C_0 and a set of j base objects $\mathcal{B}_j \subseteq \mathcal{B}$ such that, for every $\{p_0, \ldots, p_{n-j-1}\}$-only, $\mathscr{V}_j$-absent execution β from $C_0\alpha_j$, none of the base objects in $\mathcal{B}_j$ are changed during β. We now show that this holds for any $j \in \{0, \ldots, \ell\}$.

Lemma 5. *For every $j \in \{0, \ldots, \ell\}$, there is an execution α_j from C_0 and a set of j base objects $\mathcal{B}_j \subseteq \mathcal{B}$ such that*

(a) the configuration $C_0\alpha_j$ is $\{p_0, \ldots, p_{n-j-1}\}$-idle,
(b) O_k contains value 0 in $C_0\alpha_j$, and
(c) for every $\{p_0, \ldots, p_{n-j-1}\}$-only, $\mathscr{V}_j$-absent execution β from $C_0\alpha_j$, no process changes the value of any base object in $\mathcal{B}_j$ during β.

Proof. We use induction on j. Define α_0 to be the empty execution, and define $\mathcal{B}_0 = \emptyset$. Parts (a) and (b) hold trivially for $j = 0$. Part (c) also holds trivially for $j = 0$, since $\mathcal{B}_0$ is empty. Thus, the lemma holds for $j = 0$.

Now let $j \in \{0, \ldots, \ell - 1\}$ and suppose we have an execution α_j and a set of j base objects $\mathcal{B}_j$ that satisfy properties (a)-(c) of the lemma description. We will show how to construct α_{j+1} from α_j. Consider a p_0-only execution λ from $C_0\alpha_j$ that updates the values of $O_1, \ldots, O_k$ so that the object contains V_{j+1}. More formally, suppose that $V_{j+1} = (v_1, \ldots, v_k)$, and let $(o_1, \ldots, o_k)$ be the value of $[O_1, \ldots, O_k]$ in $C_0\alpha_j$. (Since $C_0\alpha_j$ is p_0-idle by part (a) of the inductive hypothesis, this value is well-defined.) Define λ as the p_0-only execution from $C_0\alpha_j$ in which, for each component i such that $o_i \neq v_i$, p_0 changes the value of component i. The operations by p_0 in λ are performed in ascending order of component index. By part (b) of the inductive hypothesis, we know that the value of O_k is 0 in $C_0\alpha_j$. Thus, the final *Update* by p_0 in λ changes the value of O_k from 0 to 1. Note that $[O_1, \ldots, O_k]$ contains the value $(v_1, \ldots, v_k) = V_{j+1}$ in $C_0\alpha_j\lambda$, O_k is the last component to be changed in λ, and all vectors in $\mathscr{V}_j$ contain 1 in their k-th component. Hence, we know that for every p_0-idle configuration C' in λ, the value contained by $[O_1, \ldots, O_k]$ in C' is not equal to any vector in the set $\mathscr{V}_j$. Thus, we know that λ is $\mathscr{V}_j$-absent (by definition of $\mathscr{V}_j$-absent).

By part (a) of the inductive hypothesis, we know that p_{n-j-1} is idle in $C_0\alpha_j$. Since p_{n-j-1} takes no steps in λ, we know that it is idle in $C_0\alpha_j\lambda$. Consider the execution δ from $C_0\alpha_j\lambda$ in which p_{n-j-1} performs a complete instance of a *Scan* operation on its own. By construction of λ, we know this *Scan* returns the vector V_{j+1}.

To derive a contradiction, suppose that:

> for every $\{p_0, \ldots, p_{n-j-1}\}$-only, $\mathscr{V}_{j+1}$-absent execution β from $C_0\alpha_j$ and for every base object $B \in \mathcal{B} - \mathcal{B}_j$, there exists a $\{p_0, \ldots, p_{n-j-2}\}$-only, $\mathscr{V}_{j+1}$-absent extension β' of β that ends with a primitive that changes the value of B so that $val(B, C_0\alpha_j\beta\beta') \neq val(B, C_0\alpha_j\beta)$. (1)

By carefully interleaving the steps of p_{n-j-1} with steps by the processes $p_0, \ldots, p_{n-j-2}$, we will show how to construct a $\mathscr{V}_{j+1}$-absent execution γ from $C_0\alpha_j$ that is indistinguishable to p_{n-j-1} from δ. This means that p_{n-j-1}'s *Scan* in γ must also return the vector V_{j+1}. Since γ is $\mathscr{V}_{j+1}$-absent (in particular, γ is V_{j+1}-absent), this contradicts Observation 3.

We will now formally show how to construct the execution γ. Suppose that δ consists of r total steps by p_{n-j-1}. For each $s \in \{0, \ldots, r\}$, let δ_s be the prefix of δ consisting of the first s steps by p_{n-j-1} (so δ_0 is the empty execution and $\delta_r = \delta$). We will use induction to show that for every $s \in \{0, \ldots, r\}$, there exists a $\{p_0, \ldots, p_{n-j-1}\}$-only, $\mathscr{V}_{j+1}$-absent execution γ_s from $C_0\alpha_j$ such that $\gamma_s \overset{p_{n-j-1}}{\sim} \delta_s$.

Let γ_0 be the empty execution. Since p_{n-j-1} takes no steps in either γ_0 or δ_0, we obtain $\gamma_0 \overset{p_{n-j-1}}{\sim} \delta_0$. Furthermore, $C_0\alpha_j$ is p_0-idle by part (a) of the inductive hypothesis and the value of O_k in $C_0\alpha_j$ is 0 by part (b) of the inductive hypothesis. Since every vector in $\mathscr{V}_{j+1}$ contains a 1 in its k-th component, γ_0 is $\mathscr{V}_{j+1}$-absent.

Now let $s \in \{1, \ldots, r\}$, and suppose we have a $\{p_0, \ldots, p_{n-j-1}\}$-only, $\mathscr{V}_{j+1}$-absent execution γ_{s-1} from $C_0\alpha_j$ such that $\gamma_{s-1} \overset{p_{n-j-1}}{\sim} \delta_{s-1}$. Since p_{n-j-1} takes no steps in λ, we have $C_0\alpha_j \overset{p_{n-j-1}}{\sim} C_0\alpha_j\lambda$. Therefore, $C_0\alpha_j\gamma_{s-1} \overset{p_{n-j-1}}{\sim} C_0\alpha_j\lambda\delta_{s-1}$. We show how to construct γ_s. Let d_s be the execution containing the s-th step applied by p_{n-j-1} during δ (i.e. $\delta_s = \delta_{s-1}d_s$). Suppose that the step by p_{n-j-1} in d_s is applied to the base object $B \in \mathcal{B}$.

First, suppose that $val(B, C_0\alpha_j\gamma_{s-1}) = val(B, C_0\alpha_j\lambda\delta_{s-1})$. Since $C_0\alpha_j\gamma_{s-1} \overset{p_{n-j-1}}{\sim} C_0\alpha_j\lambda\delta_{s-1}$, there is a p_{n-j-1}-only execution d'_s from $C_0\alpha_j\gamma_{s-1}$ in which p_{n-j-1} performs the same step as it does in d_s. We have $\gamma_s = \gamma_{s-1}d'_s \overset{p_{n-j-1}}{\sim} \delta_{s-1}d_s = \delta_s$. By Observation 4, d'_s is a $\mathscr{V}_{j+1}$-absent extension of γ_{s-1}. Thus, γ_s is $\mathscr{V}_{j+1}$-absent.

Otherwise, $val(B, C_0\alpha_j\gamma_{s-1}) \neq val(B, C_0\alpha_j\lambda\delta_{s-1})$. This means that the value of B is changed at some point during either γ_{s-1} or $\lambda\delta_{s-1}$. Since γ_{s-1} is a $\mathscr{V}_{j+1}$-absent execution by the induction hypothesis and $\mathscr{V}_j \subset \mathscr{V}_{j+1}$, we know that γ_{s-1} is also $\mathscr{V}_j$ absent. We also previously showed that λ is $\mathscr{V}_j$-absent. The execution δ_{s-1} only contains steps by the scanner p_{n-j-1}. Thus, by Observation 4, δ_{s-1} is a $\mathscr{V}_j$-absent extension of λ. Therefore, property (c) from the lemma statement implies that $B \notin \mathcal{B}_j$.

Since γ_{s-1} is a $\{p_0, \ldots, p_{n-j-1}\}$-only, $\mathscr{V}_{j+1}$-absent execution from $C_0\alpha_j$, (1) implies that there exists a $\{p_0, \ldots, p_{n-j-2}\}$-only $\mathscr{V}_{j+1}$-absent extension β' of γ_{s-1} that ends with a primitive that changes the value of B so that $val(B, C_0\alpha_j\gamma_{s-1}\beta') \neq val(B, C_0\alpha_j\gamma_{s-1})$. Since B is a 1-bit object, $val(B, C_0\alpha_j\gamma_{s-1}\beta') = val(B, C_0\alpha_j\lambda\delta_{s-1})$. Since $C_0\alpha_j\gamma_{s-1} \overset{p_{n-j-1}}{\sim} C_0\alpha_j\lambda\delta_{s-1}$ and p_{n-j-1} takes no steps in β', we have $C_0\alpha_j\gamma_{s-1}\beta' \overset{p_{n-j-1}}{\sim} C_0\alpha_j\lambda\delta_{s-1}$. Hence, there is a p_{n-j-1}-only execution d''_s from $C_0\alpha_j\gamma_{s-1}\beta'$ in which p_{n-j-1} performs the same step as it does in d_s. We know that $\gamma_s = \gamma_{s-1}\beta'd''_s \overset{p_{n-j-1}}{\sim} \delta_{s-1}d_s = \delta_s$. By Observation 4, d''_s is a $\mathscr{V}_{j+1}$-absent extension of $\gamma_{s-1}\beta'$. Thus, γ_s is $\mathscr{V}_{j+1}$-absent.

By induction, we conclude that there exists a $\{p_0, \ldots, p_{n-j-1}\}$-only, $\mathscr{V}_{j+1}$-absent execution γ_r from $C_0\alpha_j$ such that $\gamma = \gamma_r \overset{p_{n-j-1}}{\sim} \delta_r = \delta$. Since p_{n-j-1}'s *Scan* in δ returns V_{j+1} and $C_0\alpha_j \overset{p_{n-j-1}}{\sim} C_0\alpha_j\lambda$, its *Scan* in γ must also return V_{j+1}. But γ is a $\mathscr{V}_{j+1}$-absent execution (and hence, a V_{j+1}-absent execution). This contradicts Observation 3. Thus, we conclude that (1) is false, that is, there exists a $\{p_0, \ldots, p_{n-j-1}\}$-only, $\mathscr{V}_{j+1}$-absent execution σ_{j+1} from $C_0\alpha_j$ and a base object $B \in \mathcal{B} - \mathcal{B}_j$ such that for every $\{p_0, \ldots, p_{n-j-2}\}$-only, $\mathscr{V}_{j+1}$-absent extension σ' of σ_{j+1}, no process changes the value of B during σ'.

Now let τ'_{j+1} be the p_0-only execution from $C_0\alpha_j\sigma_{j+1}$ in which p_0 finishes its ongoing *Update* operation instance, if it has one. Otherwise, let τ'_{j+1} be the empty execution. Since σ_{j+1} is $\mathscr{V}_{j+1}$-absent, we know that τ'_{j+1} is a $\mathscr{V}_{j+1}$-absent extension of σ_{j+1}. Let τ''_{j+1} be the execution from $C_0\alpha_j\sigma_{j+1}\tau'_{j+1}$ in which the scanners $p_1, \ldots, p_{n-j-2}$ complete their ongoing instances of *Scan* operations in increasing order of process identifiers. By Observation 4, τ''_{j+1} is a $\mathscr{V}_{j+1}$-absent extension of $\sigma_{j+1}\tau'_{j+1}$. Define $\tau_{j+1} = \tau'_{j+1}\tau''_{j+1}$. Finally, if O_k has value 1 in $C_0\alpha_j\sigma_{j+1}\tau_{j+1}$, then define μ_{j+1} as the p_0-only execution from $C_0\alpha_j\sigma_{j+1}\tau_{j+1}$ that contains all the steps of an *Update* that changes the value of O_k to 0. Otherwise, let μ_{j+1} be the empty execution. Since every vector in $\mathscr{V}_{j+1}$ contains a 1 in its k-th component, μ_{j+1} is a $\mathscr{V}_{j+1}$-absent extension of $\sigma_{j+1}\tau_{j+1}$. Define $\alpha_{j+1} = \alpha_j\sigma_{j+1}\tau_{j+1}\mu_{j+1}$. By construction of τ_{j+1} and μ_{j+1}, we know that $C_0\alpha_{j+1}$ is $\{p_0, \ldots, p_{n-j-2}\}$-idle, and hence we have property (a) for α_{j+1}. Property (b) holds by construction of μ_{j+1}.

Define $\mathcal{B}_{j+1} = \mathcal{B}_j \cup \{B\}$, and consider some $\mathscr{V}_{j+1}$-absent execution β from $C_0\alpha_{j+1}$. We have shown that for every $\{p_0, \ldots, p_{n-j-2}\}$-only, $\mathscr{V}_{j+1}$-absent execution β from $C_0\alpha_{j+1}$, no process changes the base object B during β. Furthermore, since $\sigma_{j+1}\tau_{j+1}\mu_{j+1}$ is $\mathscr{V}_{j+1}$-absent, we know that $\sigma_{j+1}\tau_{j+1}\mu_{j+1}\beta$ is a $\{p_0, \ldots, p_{n-j-1}\}$-only, $\mathscr{V}_{j+1}$-absent execution from $C_0\alpha_j$. Since $\mathscr{V}_j \subset \mathscr{V}_{j+1}$, we know that $\sigma_{j+1}\tau_{j+1}\mu_{j+1}\beta$ is also $\mathscr{V}_j$-absent. Thus, by property (c) of the inductive hypothesis, no process changes any base object in $\mathcal{B}_j$ during β. We have therefore shown that property (c) holds for $\mathcal{B}_{j+1}$. This concludes the construction of α_{j+1}. □

We now show that at least $k + 1$ base objects are required by the implementation $\mathcal{I}$. We first apply Lemma 5 to obtain a base object B and an execution such that, in any V_1-absent extension of this execution that does not involve p_1, the base object B does not change. If the implementation has only k base objects, then there are only $k - 1$ base objects other than B. However, during a V_1-absent execution, it must be possible for p_0 to represent any k-component binary vector besides V_1 in the scannable object. A simple information theory argument shows that this is not possible with only $k - 1$ usable base objects.

Lemma 6. *The number of base objects m required by the implementation $\mathcal{I}$ is at least $k+1$.*

Proof. To obtain a contradiction, assume $m = k$. Since $n, k \geq 2$, we have $\ell = min(n-1, 2^{k-1}) \geq 1$. By Lemma 5, there is an execution α_1 from C_0 and a base object $B \in \mathcal{B}$ such that

(a) p_0 is idle in $C_0\alpha_1$,
(b) O_k contains value 0 in $C_0\alpha_1$, and
(c) for every $\{p_0, \ldots, p_{n-2}\}$-only, V_1-absent execution β from $C_0\alpha_1$, no process changes the value of B during β.

Let $\{U_1, \ldots, U_{2^k-1}\}$ be the set of all k-component binary vectors except V_1. Let $U_j = (u_{j,1}, \ldots, u_{j,k})$ for all $j \in \{1, \ldots, 2^k - 1\}$. Let $(o_1, \ldots, o_k)$ be value of $[O_1, \ldots, O_k]$ in $C_0\alpha_1$. By (a), we know that p_0 is idle in $C_0\alpha_1$. For each $j \in \{1, \ldots, 2^k - 1\}$, define λ_j as the p_0-only execution from $C_0\alpha_1$ in which, for each component i such that such that $o_i \neq u_{j,i}$, p_0 changes O_i to $u_{j,i}$. The operations by p_0 in λ_j are performed in ascending order of i. By (b), we know that $o_k = 0$. Thus, if $u_{j,k} = 1$, the final instance of an *Update* operation by p_0 in λ_j changes the value of O_k from 0 to 1. Since V_1 contains a 1 in every component and $U_j \neq V_1$, the value of $[O_1, \ldots, O_k]$ is not equal to V_1 in any p_0-idle configuration in λ_j. Then λ_j is V_1-absent (by definition of V_1-absent). Hence, by (c), $val(B, C_0\alpha_1\lambda_j) = val(B, C_0\alpha_1)$. Since $m = k = |\mathcal{B}|$, we have $|\mathcal{B} - \{B\}| = k - 1$.

Since $k \geq 2$, it follows that $2^{k-1} < 2^k - 1 = |\{U_1, \ldots, U_{2^k-1}\}|$. Since each object in $\mathcal{B} - \{B\}$ can only contain a 0 or a 1, it follows by the pigeonhole principle that there are two distinct indices $a, b \in \{1, \ldots, 2^k - 1\}$ such that all the base objects have the same value in $C_0\alpha_1\lambda_a$ and $C_0\alpha_1\lambda_b$. Process p_{n-1} is either idle or it has an ongoing instance of a *Scan* operation in $C_0\alpha_1$. Since λ_a is a p_0-only execution, p_{n-1} is still either idle or has an ongoing *Scan* in $C_0\alpha_1\lambda_a$. Define δ_a as the p_{n-1}-only execution from $C_0\alpha_1\lambda_a$ in which p_{n-1} completes its ongoing *Scan* (if it has one), and then performs another complete instance sc of a *Scan* operation. By construction of λ_a, we know that sc must return the vector U_a. However, since λ_b is also p_0-only, we know that $C_0\alpha_1\lambda_a \overset{p_{n-1}}{\sim} C_0\alpha_1\lambda_b$. Furthermore, all of the base objects have the same value in $C_0\alpha_1\lambda_a$ and $C_0\alpha_1\lambda_b$, so there is an execution δ_b from $C_0\alpha_1\lambda_b$ such that $\delta_a \overset{p_{n-1}}{\sim} \delta_b$. Thus, the instance of a *Scan* operation that is started by p_{n-1} during δ_b must return U_a as well. But this is a contradiction, because the *Scan* that p_{n-1} starts in its solo execution from $C_0\alpha_1\lambda_b$ must return the vector $U_b \neq U_a$ by construction of λ_b. We conclude that $m \geq k + 1$. □

Using Lemma 5, we will identify a configuration and a set of $h = min(n-2, 2^{k-1})$ base objects such that any execution from this configuration (not involving the last h scanners) does not change any of these base objects, provided p_0 does not perform any instances of *Update* operations on component k. Therefore, the remaining $n - h$ processes (including p_0) essentially implement a scannable binary object with $k - 1$ components starting from this configuration using the other $n - h$ base objects. We formalize this argument in Lemma 7.

Lemma 7. *If $k \geq 3$, then there is an obstruction-free, linearizable, single-updater, $(n-h)$-process implementation $\mathcal{I}'$ of the scannable binary object $[O_1, \ldots, O_{k-1}]$ from $m-h$ instances of 1-bit base objects, where $h = min(n-2, 2^{k-1})$.*

Proof. Notice that $h \leq \ell$. By Lemma 5, there is an execution α_h from C_0 and a set of h base objects $\mathcal{B}_h \subseteq \mathcal{B}$ such that

(a) the processes $p_0, \ldots, p_{n-h-1}$ are all idle in $C_0\alpha_h$,
(b) O_k contains value 0 in $C_0\alpha_h$, and
(c) for every $\{p_0, \ldots, p_{n-h-1}\}$-only, $\mathcal{V}_h$-absent execution β from $C_0\alpha_h$, no process changes the value of any base object in $\mathcal{B}_h$ during β.

Note that p_0 is idle in $C_0\alpha_h$ by (a). Let κ be the p_0-only execution from $C_0\alpha_h$ in which p_0 changes the value of O_i to 0 (if O_i contains the value 1 in $C_0\alpha_h$) in ascending order of i. Since κ is a p_0-only execution and all of the processes $p_0, \ldots, p_{n-h-1}$ are idle in $C_0\alpha_h$ by (a), they are all idle in $C_0\alpha_h\kappa$ as well. Let $D_0 = C_0\alpha_h\kappa$. Then the implementation $\mathcal{I}'$ is obtained by simulating steps of the processes $p_0, \ldots, p_{n-h-1}$ on $[O_1, \ldots, O_{k-1}]$ starting from the configuration D_0. The base objects $\mathcal{B}' = \{B'_0, \ldots, B'_{n-h-1}\}$ used by $\mathcal{I}'$ are copies of the base objects in $\mathcal{B} - \mathcal{B}_h$. Let B_j be the base object in $\mathcal{B} - \mathcal{B}_h$ that corresponds to B'_j and let the initial value of B'_j be the value of B_j in D_0. Let $q_0, \ldots, q_{n-h-1}$ be the processes participating in $\mathcal{I}'$, where q_j stores the local state of p_j and simulates the steps of p_j. In the initial configuration, C'_0, each process q_j stores the state p_j has in D_0.

When q_j is given an instance of an *Update* or *Scan* operation on $\mathcal{I}'$ by the scheduler, it simulates the same operation on $\mathcal{I}$ by p_j. Since $\mathcal{I}'$ is an implementation of a $(k-1)$-component object, the scheduler only issues instances of $Update(i, op)$ operations for $i \in \{1, \ldots, k-1\}$. Therefore, q_0 never simulates any instances of *Update* operations by p_0 on O_k. Furthermore, component O_k contains the value 0 in configuration D_0 by property (b) and by the definition of κ. Since every vector in $\mathcal{V}_h$ has a 1 in its k-th component, the simulated execution of $\mathcal{I}$ is guaranteed to be $\mathcal{V}_h$-absent.

Whenever q_j is scheduled to take a step, it does the following. If p_j is poised to apply a primitive to the base object $B_r \in \mathcal{B} - \mathcal{B}_h$, then q_j applies the same primitive to B'_r and updates its copy of p_j's local state based on the response of this step. If p_j is poised to apply a primitive to some base object in $\mathcal{B}_h$, process q_j locally simulates steps of p_j until either p_j is poised to apply a primitive to some base object in $\mathcal{B} - \mathcal{B}_h$, or p_j's operation instance responds. (By property (c), none of the simulated processes change any of the base objects in $\mathcal{B}_h$ in the simulated execution. Therefore, process q_j simply ignores any step in which p_j is poised to apply a primitive that does not return a value to a base object in $\mathcal{B}_h$. If the primitive does return a value, q_j can simulate this step by treating its response to be the value of this base object in configuration D_0.) When p_0 returns a response to an instance of an $Update(i, op)$ operation, q_0 returns with the same response. Finally, when p_j returns a response to a *Scan*, q_j returns with a response consisting of the first $k-1$ components of the vector returned by p_j.

To show that the simulation is correct, consider an arbitrary execution τ' of $\mathcal{I}'$ starting from C'_0. Let τ be the execution of $\mathcal{I}$ from D_0 constructed iteratively as follows: For each step t of τ' by process q_j, schedule p_j to take the steps simulated by q_j during t. This may be more than one step if q_j locally simulated some steps applied to base objects in $\mathcal{B}_h$. Notice that τ and τ' contain exactly the same instances of operations and responses on the implemented object in exactly the same order. (Note that the response to each

Scan in τ' contains only the first $k-1$ components of the response to the simulated *Scan* in τ. However, this distinction is not important and we ignore it below.)

Since $\mathcal{I}$ is linearizable, there exists a linearization Λ of the execution $\alpha_h\kappa\tau$. Let Π be the sequence obtained from Λ by removing all of the pairs containing operations from $\alpha_h\kappa$. We know that Π satisfies the sequential specification of $[O_1, \ldots, O_k]$. Furthermore, Π contains all of the complete instances of operations from τ immediately followed by their responses in τ, along with some subset of the incomplete instances of operations immediately followed by some responses. Finally, we know that every pair of non-overlapping operation instances in τ have the same relative order in Π. Since τ and τ' contain exactly the same instances of operations and responses on the implemented object in exactly the same order, we know that Π is a linearization of τ'. Hence, $\mathcal{I}'$ is linearizable.

□

So far, we have considered an arbitrary obstruction-free, linearizable, single-updater, n-process implementation $\mathcal{I}$ of a scannable binary object $[O_1, \ldots, O_k]$, where each component is non-monotonic and $n, k \geq 2$. We now begin a new scope for the variables n, k, and $\mathcal{I}$. That is, we reuse the variables n, k, and $\mathcal{I}$ below. We will apply Lemma 6 and Lemma 7 to multiple different implementations (for different quantities of processes and components) in the proof of Theorem 8.

We will now present our main result. Assume we have an implementation of an n-process scannable binary object with k components using m instances of 1-bit base objects, Lemma 7 implies that there is a $(k-1)$-component implementation that works for $h = min(n-2, 2^{k-1})$ fewer processes and uses h fewer base objects. We can repeatedly apply this until we have obtained a 2-process implementation of an object with $k' < k$ components. Then, using Lemma 6, we know that there must be at least $k'+1$ base objects remaining. This allows us to obtain a lower bound on the number of base objects in the original k-component implementation. We formalize this with an inductive argument in Theorem 8.

Theorem 8. *Let $r \geq 0$. For all k, n such that $k \geq r+3$ and $2^k - 2^{k-r} < n-2 \leq 2^k - 2^{k-r-1}$, any obstruction-free, linearizable, single-updater, n-process implementation $\mathcal{I}$ of a scannable binary object $[O_1, \ldots, O_k]$, where each component O_i is non-monotonic, requires at least $n+k-r-2$ instances of 1-bit base objects.*

Proof. We use induction on r. When $r = 0$, we have $k \geq 3$ and $0 < n-2 \leq 2^{k-1}$. Suppose that there is an obstruction-free, linearizable, single-updater, n-process implementation $\mathcal{I}$ of $[O_1, \ldots, O_k]$ from m instances of 1-bit base objects. By Lemma 7 (with $h = n-2$), there is an obstruction-free, linearizable, single-updater, 2-process implementation $\mathcal{I}'$ of the scannable binary object $[O_1, \ldots, O_{k-1}]$ from $m-n+2$ instances of 1-bit base objects. By Lemma 6, we have $m-n+2 \geq k$, which means that $m \geq n+k-2 = n+k-r-2$.

Now let $r > 0$, and suppose that the lemma holds for $r-1$. Assume that, for some k, n such that $k \geq r+3$ and $2^k - 2^{k-r} < n-2 \leq 2^k - 2^{k-r-1}$, there is an obstruction-free, linearizable, single-updater, n-process implementation $\mathcal{I}$ of $[O_1, \ldots, O_k]$ from m instances of 1-bit base objects. By Lemma 7 (with $h = 2^{k-1}$), there is an obstruction-free, linearizable, single-updater, $(n-2^{k-1})$-process implementation $\mathcal{I}'$ of the scannable binary object $[O_1, \ldots, O_{k-1}]$ from $m-2^{k-1}$ instances of 1-bit base objects. Notice that $k-1 \geq (r-1)+3$ and $2^{k-1} - 2^{k-r} < n-2^{k-1}-2 \leq 2^{k-1} - 2^{k-r-1}$. Then by the inductive hypothesis, we know that $m-2^{k-1} \geq n-2^{k-1}+(k-1)-(r-1)-2$. Hence, $m \geq n+k-r-2$. □

Lemma 6 gives us Corollary 9 (a). Taking $r = 0$, Theorem 8 gives us Corollary 9 (b). This nearly matches our upper bound of $n+k$ base objects from Section 4.1. Taking $r = k-3$ and $n = 2^k - 2$, Theorem 8 implies that an n-process implementation of a scannable binary object with k non-monotonic components requires at least $2^k - 1$ base objects. This lower bound also applies to larger values of n. When $k = 2$, Lemma 6 implies that $m \geq k+1 = 2^k - 1$. This gives us Corollary 9 (c). This nearly matches our single-updater upper bound of 2^k base objects from Section 4.2.

Corollary 9. *For all $k \geq 3$, any obstruction-free, linearizable, n-process implementation of a scannable binary object $[O_1, \ldots, O_k]$, where each component O_i is non-monotonic, requires at least*

(a) $k+1$ instances of 1-bit base objects when $n = 2$,
(b) $n+k-2$ instances of 1-bit base objects when $2 < n \leq 2^{k-1}+2$, and
(c) $2^k - 1$ instances of 1-bit base objects when $n \geq 2^k - 2$.

6 CONCLUSION

Our lower bound of $n+k-2$ for single-updater obstruction-free implementations, which holds when $2 \leq n \leq 2^{k-1}+2$, almost matches our $n+k$ upper bound for lock-free implementations where every process can *Scan* and *Update*. This shows that, for this range of n, a single-updater or obstruction-free implementation cannot be much more efficient. For all values of n, we conjecture that $n+k$ single-bit base objects are necessary for any n-process implementation of a scannable binary object with k non-monotonic components in which every process can perform *Scans* and *Updates*.

Initially, our proofs in Section 5 needed every process to do both *Update* and *Scan* operations. Ultimately we realized that our arguments could be applied in the single-updater setting, which both strengthens the result and simplifies the proofs. The single-scanner implementation in Section 4.1 from $k+1$ base objects and the single-updater implementation in Section 4.2 from 2^k base objects mean that, to extend our conjectured lower bound to larger values of n, we will need to consider more complex executions that contain both overlapping *Update* and overlapping *Scan* operations.

It may also be possible to improve our lower bound by using a stronger progress condition such as lock-freedom or wait-freedom. Another direction is to extend our work to scannable objects whose components have larger domains.

ACKNOWLEDGMENTS

I thank my supervisor, Faith Ellen, for all of the helpful discussions, proofreading, and guidance over the course of this project. I also thank the anonymous reviewers for their comments. Support is gratefully acknowledged from the Natural Science and Engineering Research Council of Canada under grant RGPIN-2020-04178. I also gratefully acknowledge the support of the Ontario Graduate Scholarship (OGS) Program.

REFERENCES

[1] Yehuda Afek, Hagit Attiya, Danny Dolev, Eli Gafni, Michael Merritt, and Nir Shavit. 1993. Atomic Snapshots of Shared Memory. *J. ACM* 40, 4 (Sept. 1993), 873–890. https://doi.org/10.1145/153724.153741

[2] James Anderson. 1990. Composite Registers. In *Proceedings of the Ninth Annual ACM Symposium on Principles of Distributed Computing* (Quebec City, Quebec, Canada) *(PODC '90)*. Association for Computing Machinery, New York, NY, USA, 15–29. https://doi.org/10.1145/93385.93396

[3] James H. Anderson. 1994. Multi-writer composite registers. *Distributed Computing* 7, 4 (01 May 1994), 175–195. https://doi.org/10.1007/BF02280833

[4] James Aspnes, Hagit Attiya, and Keren Censor. 2009. Max Registers, Counters, and Monotone Circuits. In *Proceedings of the 28th ACM Symposium on Principles of Distributed Computing* (Calgary, AB, Canada) *(PODC '09)*. Association for Computing Machinery, New York, NY, USA, 36–45. https://doi.org/10.1145/1582716.1582728

[5] James Aspnes, Hagit Attiya, Keren Censor-Hillel, and Faith Ellen. 2015. Limited-Use Atomic Snapshots with Polylogarithmic Step Complexity. *J. ACM* 62, 1 (2015), 3:1–3:22. https://doi.org/10.1145/2732263

[6] J. Aspnes and M. Herlihy. 1990. Wait-Free Data Structures in the Asynchronous PRAM Model. In *Proceedings of the Second Annual ACM Symposium on Parallel Algorithms and Architectures* (Island of Crete, Greece) *(SPAA '90)*. Association for Computing Machinery, New York, NY, USA, 340–349. https://doi.org/10.1145/97444.97701

[7] Hagit Attiya and Faith Ellen. 2014. *Impossibility Results for Distributed Computing*. Morgan & Claypool Publishers. https://doi.org/10.2200/S00551ED1V01Y201311DCT012

[8] Hagit Attiya, Maurice Herlihy, and Ophir Rachman. 1995. Atomic snapshots using lattice agreement. *Distributed Computing* 8, 3 (01 Mar 1995), 121–132. https://doi.org/10.1007/BF02242714

[9] Hagit Attiya and Ophir Rachman. 1998. Atomic Snapshots in O(n log n) Operations. *SIAM J. Comput.* 27, 2 (1998), 319–340. https://doi.org/10.1137/S0097539795279463

[10] Hagit Attiya and Jennifer L. Welch. 2004. *Distributed computing - fundamentals, simulations, and advanced topics (2. ed.)*. Wiley.

[11] Zohir Bouzid, Michel Raynal, and Pierre Sutra. 2018. Anonymous obstruction-free (n, k)-set agreement with n-k+1 atomic read/write registers. *Distributed Comput.* 31, 2 (2018), 99–117. https://doi.org/10.1007/s00446-017-0301-7

[12] J.E. Burns and N.A. Lynch. 1993. Bounds on Shared Memory for Mutual Exclusion. *Information and Computation* 107, 2 (1993), 171 – 184. https://doi.org/10.1006/inco.1993.1065

[13] Faith Ellen, Panagiota Fatourou, and Eric Ruppert. 2007. Time Lower Bounds for Implementations of Multi-Writer Snapshots. *J. ACM* 54, 6 (Dec. 2007), 30–es. https://doi.org/10.1145/1314690.1314694

[14] Faith Ellen, Panagiota Fatourou, and Eric Ruppert. 2008. The space complexity of unbounded timestamps. *Distributed Computing* 21, 2 (01 Jul 2008), 103–115. https://doi.org/10.1007/s00446-008-0060-6

[15] Faith Ellen, Rati Gelashvili, Nir Shavit, and Leqi Zhu. 2020. A complexity-based classification for multiprocessor synchronization. *Distributed Computing* 33, 2 (01 Apr 2020), 125–144. https://doi.org/10.1007/s00446-019-00361-3

[16] Panagiota Fatourou and Nikolaos D. Kallimanis. 2017. Lower and upper bounds for single-scanner snapshot implementations. *Distributed Computing* 30, 4 (01 Aug 2017), 231–260. https://doi.org/10.1007/s00446-016-0286-7

[17] Faith Fich and Eric Ruppert. 2003. Hundreds of impossibility results for distributed computing. *Distributed Computing* 16, 2 (01 Sep 2003), 121–163. https://doi.org/10.1007/s00446-003-0091-y

[18] Rainer Gawlick, Nancy A. Lynch, and Nir Shavit. 1992. Concurrent Timestamping Made Simple. In *Theory of Computing and Systems, ISTCS'92, Israel Symposium, Haifa, Israel, May 1992 (Lecture Notes in Computer Science, Vol. 601)*, Danny Dolev, Zvi Galil, and Michael Rodeh (Eds.). Springer, 171–183. https://doi.org/10.1007/BFb0035176

[19] Rachid Guerraoui and Petr Kuznetsov. 2018. *Algorithms for Concurrent Systems*. EPFL press.

[20] Rachid Guerraoui and Eric Ruppert. 2007. Anonymous and fault-tolerant shared-memory computing. *Distributed Comput.* 20, 3 (2007), 165–177. https://doi.org/10.1007/s00446-007-0042-0

[21] Maryam Helmi, Lisa Higham, Eduardo Pacheco, and Philipp Woelfel. 2014. The Space Complexity of Long-Lived and One-Shot Timestamp Implementations. *J. ACM* 61, 1, Article 7 (Jan. 2014), 25 pages. https://doi.org/10.1145/2559904

[22] Maurice Herlihy. 1991. Wait-Free Synchronization. *ACM Trans. Program. Lang. Syst.* 13, 1 (Jan. 1991), 124–149. https://doi.org/10.1145/114005.102808

[23] Jaap-Henk Hoepman and John Tromp. 1993. Binary Snapshots. In *Distributed Algorithms, 7th International Workshop, WDAG '93, Lausanne, Switzerland, September 27-29, 1993, Proceedings (Lecture Notes in Computer Science, Vol. 725)*, André Schiper (Ed.). Springer, 18–25. https://doi.org/10.1007/3-540-57271-6_24

[24] Michiko Inoue and Wei Chen. 1994. Linear-Time Snapshot Using Multi-writer Multi-reader Registers. In *Distributed Algorithms, 8th International Workshop, WDAG '94, Terschelling, The Netherlands, September 29 - October 1, 1994, Proceedings (Lecture Notes in Computer Science, Vol. 857)*, Gerard Tel and Paul M. B. Vitányi (Eds.). Springer, 130–140. https://doi.org/10.1007/BFb0020429

[25] A. Israeli, A. Shaham, and A. Shirazi. 1995. Linear-time snapshot implementations in unbalanced systems. *Mathematical systems theory* 28, 5 (01 Sep 1995), 469–486. https://doi.org/10.1007/BF01185868

[26] Prasad Jayanti. 2002. F-Arrays: Implementation and Applications. In *Proceedings of the Twenty-First Annual Symposium on Principles of Distributed Computing* (Monterey, California) *(PODC '02)*. Association for Computing Machinery, New York, NY, USA, 270–279. https://doi.org/10.1145/571825.571875

[27] Prasad Jayanti. 2005. An Optimal Multi-Writer Snapshot Algorithm. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing* (Baltimore, MD, USA) *(STOC '05)*. Association for Computing Machinery, New York, NY, USA, 723–732. https://doi.org/10.1145/1060590.1060697

[28] Prasad Jayanti, King Tan, and Sam Toueg. 2000. Time and Space Lower Bounds for Nonblocking Implementations. *SIAM J. Comput.* 30, 2 (April 2000), 438–456. https://doi.org/10.1137/S0097539797317299

[29] L. M. Kirousis, P. Spirakis, and P. Tsigas. 1994. Reading many variables in one atomic operation: solutions with linear or sublinear complexity. *IEEE Transactions on Parallel and Distributed Systems* 5, 7 (1994), 688–696. https://doi.org/10.1109/71.296315

[30] Gary L. Peterson. 1983. Concurrent Reading While Writing. *ACM Trans. Program. Lang. Syst.* 5, 1 (Jan. 1983), 46–55. https://doi.org/10.1145/357195.357198

[31] Yaron Riany, Nir Shavit, and Dan Touitou. 2001. Towards a practical snapshot algorithm. *Theoretical Computer Science* 269, 1 (2001), 163 – 201. https://doi.org/10.1016/S0304-3975(00)00412-6

[32] K. Vidyasankar. 1988. Converting Lamport's Regular Register to Atomic Register. *Inf. Process. Lett.* 28, 6 (Aug. 1988), 287–290. https://doi.org/10.1016/0020-0190(88)90175-5

[33] Yuanhao Wei, Naama Ben-David, Guy E. Blelloch, Panagiota Fatourou, Eric Ruppert, and Yihan Sun. 2021. Constant-Time Snapshots with Applications to Concurrent Data Structures. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Virtual Event, Republic of Korea) *(PPoPP '21)*. Association for Computing Machinery, New York, NY, USA, 31–46. https://doi.org/10.1145/3437801.3441602

[34] Leqi Zhu. 2016. A tight space bound for consensus. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2016, Cambridge, MA, USA, June 18-21, 2016*, Daniel Wichs and Yishay Mansour (Eds.). ACM, 345–350. https://doi.org/10.1145/2897518.2897565

[35] Leqi Zhu. 2019. *On the Space Complexity of Colourless Tasks*. Ph.D. Dissertation. University of Toronto.

[36] Leqi Zhu and Faith Ellen. 2015. Atomic Snapshots from Small Registers. In *19th International Conference on Principles of Distributed Systems, OPODIS 2015, December 14-17, 2015, Rennes, France (LIPIcs, Vol. 46)*, Emmanuelle Anceaume, Christian Cachin, and Maria Gradinariu Potop-Butucaru (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 17:1–17:16. https://doi.org/10.4230/LIPIcs.OPODIS.2015.17